Credit-Scoring-Engine Curriculum

Credit-Scoring-Engine — Chaptered Rebuild Curriculum

Purpose: Walk the existing pipeline stage by stage. Each chapter covers three tracks — Data Science Fundamentals, Statistics/Math/Finance, and Python — with real explanation, not just topic labels, grounded in the actual credit-scoring-engine project.

How to use it: Read a chapter fully, then attempt to reason through that stage yourself before opening the real notebook. Use the notebooks and docs (EDA_summary.md, PRD-CreditScoringEngine.md) as an answer key to

check yourself afterward, not as a starting point. Each chapter ends with a short “Try this yourself” prompt — do it before moving on.

Honest caveat: This is the slower, deeper path compared to a rehearsal project or jumping straight into new development. Expect real time investment if you do this properly. Also, you’ve already seen the EDA and model results from earlier work on this project, so this is closer to verifying you can now derive what’s already there than a blind first discovery.

Chapter 0 — Problem Framing (before importing the CSV)

a. Data Science Fundamentals

This project is a supervised binary classification problem. “Supervised” means every training example already carries the correct answer — in your dataset that’s the TARGET column, 1 if the applicant eventually defaulted, 0 if they repaid. “Binary” means there are only two possible answers, not a spectrum. Before writing any code, be precise about what a “feature” is: any column other than TARGET and the applicant ID (SK_ID_CURR) is information about that applicant that might correlate with their eventual outcome.

The most important discipline at this stage is distinguishing training data from production data. Everything in application_train.csv already knows the outcome, because these are closed cases. A real applicant walking into a branch today does not have a TARGET value — that is exactly what the model exists to predict. Any feature that could only be known after the loan was disbursed and time had passed (a payment history field, a “was flagged for review” field) is a leakage risk, because it would not be available for the real prediction task. This is one of the most expensive mistakes to make early, because it silently makes a model look far better in testing than it will ever perform in reality.

b. Statistics / Math / Finance

307,511 rows is a sample, not a population — one company's applicants over one period, not every borrower everywhere for all time. Every conclusion drawn from it is technically an inference about a broader population, and that inference gets shakier the more real-world applicants differ from this sample.

Before touching any code, internalize the base rate: 24,825 of 307,511 applicants defaulted, which is 8.07%. This single number should already change how you think about every later step. A lazy model that predicts "no default" for everyone would be "right" 91.93% of the time while catching zero actual defaulters — which is exactly why accuracy gets abandoned as a metric later.

You also need working financial vocabulary before the feature names make sense. AMT_CREDIT is the amount of credit granted (the loan principal). AMT_GOODS_PRICE is the price of the item being financed (often a car or appliance) and is not always equal to AMT_CREDIT — the gap between the two is itself informative, since financing more than the goods are worth is a risk signal. AMT_ANNUITY is the fixed periodic repayment amount. "Default" means the borrower failed to meet the agreed repayment terms.

c. Python

At this stage you need the vocabulary of tabular data, not modeling code. A DataFrame is pandas' representation of a table — rows and named columns, similar to a spreadsheet but with types tracked per column. A Series is a single column pulled out of a DataFrame. `pd.read_csv('application_train.csv')` loads the file into a DataFrame.

Once loaded, `.shape` gives you (rows, columns) as a quick sanity check against the known 307,511 x 122. `.head()` shows the first few rows so you can look at what the data actually contains rather than trusting a description of it. `.info()` shows the column list with data types and non-null counts in one pass. `.dtypes` isolates just the type of each column — a column like CODE_GENDER stored as text (object) versus DAYS_BIRTH stored as a number changes what operations even make sense on it.

Try this yourself: Before opening 01_eda.ipynb, write down what you expect `.info()` to show for AMT_INCOME_TOTAL's dtype and for

CODE_GENDER’s dtype. Then check.

Chapter 1 — Data Loading & First Inspection

a. Data Science Fundamentals

First inspection is not the same activity as EDA — it is a sanity check, not analysis. Its goal is to catch anything that would make later analysis wrong before you have invested time in it. Concretely: confirm the row and column counts match what you expect (307,511 x 122 here); identify which columns are identifiers (SK_ID_CURR) rather than features, since an ID column has no predictive meaning and treating it as one would be a bug, not a discovery; and take a first look at the target column itself, confirming it really is binary and checking raw counts of each class before computing any percentage, since a computed percentage can hide whether you are looking at 25,000 positives or 25.

This is also the stage where a genuinely broken value would first show up if you were paying attention — DAYS_EMPLOYED, for instance, would reveal a suspicious maximum at this very first look (Chapter 2 explains why).

b. Statistics / Math

This is where descriptive statistics get used for the first time — and only descriptive, not inferential, because you are describing the sample in front of you, not yet generalizing beyond it. For every numerical column you want the mean, median, mode, and a sense of spread (range, standard deviation, interquartile range).

The reason to look at more than the mean alone is that a single summary number can mislead you: if a column’s mean is far from its median, that is your first clue the column is skewed — exactly the pattern that applies to AMT_INCOME_TOTAL in this project, well before Chapter 2’s deeper dive into it. Getting comfortable reading a `.describe()` table — recognizing which columns have a suspiciously large max relative to their 75th

percentile — is what turns this from a mechanical step into a real inspection.

c. Python

`.describe()` gives count, mean, std, min, quartiles, and max for every numerical column in one call — the fastest way to spot the "suspiciously large max" pattern above. `.nunique()` gives the number of distinct values a column has, which is how you tell a categorical column with 5 options apart from one with 500. `.isnull().sum()` gives a raw count of missing values per column — the necessary input to Chapter 2's tiered missingness strategy. Basic column selection (`df['AMT_INCOME_TOTAL']`, `df[['AMT_INCOME_TOTAL', 'AMT_CREDIT']]`) lets you inspect one thing at a time instead of only ever looking at the whole table.

Try this yourself: Before running `.isnull().sum()`, guess which three columns you would expect to have the most missing values, based only on the feature names in feature_list.txt. Then check your guess.

Chapter 2 — EDA: Distributions, Missingness, Imbalance, Correlation

a. Data Science Fundamentals

EDA exists to shape every decision that follows it — you do not pick an imputation strategy, an encoding scheme, or an evaluation metric before understanding the data; you pick them because of what EDA showed you. Class imbalance is the clearest example here: the 8.07% default rate is the reason scale_pos_weight shows up in the LightGBM training code later, not a footnote to skip past.

Missingness is the second major finding, and it needs a real strategy rather than one blanket rule — 67 of 122 columns have some missing data, and treating a column missing 0.4% of the time the same as one missing 70% of the time would be a mistake in either direction. And this project has a

textbook example of why you inspect data instead of trusting it: DAYS_EMPLOYED contains the value 365,243 for 55,374 applicants (18% of the dataset) — a sentinel value some upstream system used to mean "not applicable" (pensioners and the unemployed), not an actual day count. Nobody would guess that from a data dictionary; it only shows up when you actually look at the distribution.

b. Statistics / Math

Skewness connects several EDA findings together: when a distribution has a long tail on one side — income is the clear case here, right-skewed, with a small number of very high earners pulling the mean well above the median — the mean stops representing anything "typical," which is exactly why median was chosen for central tendency on financial columns.

Correlation quantifies linear relationship strength between two numerical variables on a -1 to +1 scale. This project's EDA surfaced EXT_SOURCE_3 (0.1789), EXT_SOURCE_2 (0.1605), and EXT_SOURCE_1 (0.1553) as the strongest individual numerical predictors of default — worth noting these are external credit bureau scores, not raw application data, which is itself informative about what actually predicts risk. Equally important: correlation is not causation. Income and loan amount may move together without income directly causing the specific loan size chosen.

Empirical probability is how the 8.07% base rate itself was derived — by counting outcomes directly in observed data, not from a formula. The missingness tiering (low under 10%, medium 10-50%, high over 50%) is itself a statistical judgment call about how much you can trust an imputed value to stand in for a real one as more of a column goes missing.

c. Python

matplotlib and seaborn are the workhorses: histograms to see a distribution's shape directly (how you would actually see income's right skew, not just infer it from summary numbers), boxplots to spot outliers and compare a numerical variable across groups, and heatmaps to visualize a correlation matrix at a glance. `df.corr()` computes pairwise correlation across all numerical columns in one call. `df.isnull().mean()` turns Chapter

1’s raw missing counts into percentages, feeding directly into the tiering decision.

Try this yourself: Without opening the notebook, predict whether you would expect DAYS_EMPLOYED’s anomalous group (the 365,243 sentinel rows) to have a higher or lower default rate than everyone else, and why. Then check against the actual numbers (5.4% versus 8.66%) — did your reasoning point the right direction?

Chapter 3 — Feature Engineering & Encoding

a. Data Science Fundamentals

Every column falls into one of four types: continuous numerical (income, credit amount), discrete numerical (number of children), nominal categorical (contract type, with no inherent order), or ordinal categorical (education level, which has a real order). That classification decides how you are allowed to encode it, not personal preference.

Feature engineering is the practice of constructing new columns that make an existing relationship easier for a model to use directly, rather than hoping the model rediscovers it from raw numbers. This project’s debt-to-income and annuity-to-credit ratios exist because raw income or credit alone showed only a weak, non-linear relationship with default, but the ratio between two related numbers captured the signal far more directly. This is also a natural point to revisit leakage: a derived feature is only safe if every input it is built from would genuinely be known at prediction time — DEBT_TO_INCOME qualifies because both AMT_CREDIT and AMT_INCOME_TOTAL are known at application time.

b. Statistics / Math / Finance

DEBT_TO_INCOME (AMT_CREDIT divided by AMT_INCOME_TOTAL) directly answers “how large is this loan relative to what the applicant earns” — a much more directly risk-relevant question than either number alone. ANNUITY_TO_INCOME (AMT_ANNUITY divided by AMT_INCOME_TOTAL)

captures repayment burden as a fraction of income, closer to how a loan officer would actually reason than an absolute amount. CREDIT_TO_GOODS (AMT_CREDIT divided by AMT_GOODS_PRICE) flags financing that exceeds the value of the underlying goods, a classic risk signal in lending.

Log transformation (AMT_INCOME_TOTAL_LOG) compresses the long right tail of a skewed variable, which particularly helps a linear model like logistic regression, whose baseline coefficients assume roughly linear relationships — a mathematical reason, not just a code habit.

c. Python

`pd.get_dummies()` performs one-hot encoding, turning a nominal categorical column into a set of binary 0/1 columns, one per category — the right choice for something like NAME_HOUSING_TYPE, which has no natural order. For higher-cardinality columns like OCCUPATION_TYPE or ORGANIZATION_TYPE, one-hot encoding would explode into dozens of sparse columns, so this project's docs describe target encoding instead — replacing each category with a number derived from its historical default rate, typically implemented as a grouped mean via pandas rather than one-hot. `np.log1p()` (log of 1+x, which safely handles zero values) is the standard call for log-transforming a skewed column like income.

Try this yourself: NAME_EDUCATION_TYPE has a real order (Primary < Secondary < Higher). Would one-hot encoding lose that ordering information? What would you use instead, and why does that choice matter more for a linear model than for a tree-based one?

Chapter 4 — Train/Validation/Test Split & Imputation

a. Data Science Fundamentals

Splitting comes before imputation and scaling, not after, for a specific reason: if you compute a median or mean using the full dataset and then use

it to fill missing values everywhere, information from your validation and test sets has leaked into training — you would be partly evaluating the model on knowledge it was not supposed to have.

Three sets rather than two exist because you need one set to make decisions during development (validation) and one completely untouched set to report a final, honest number (test) — if you tune anything based on test performance, test stops being a fair measure. Stratification means preserving the original class ratio in every split rather than letting it drift randomly — worth checking directly in this project's own split code, since with only an 8.07% positive rate, an unstratified split risks a validation set with a meaningfully different, misleading default rate.

b. Statistics / Math

This is sampling logic applied directly: splitting 307,511 rows into three groups purely at random, without constraining the class ratio, means each subset's default rate will wobble around 8.07% rather than matching it exactly through ordinary sampling variance — usually not by much at this sample size, but that is exactly the kind of assumption worth verifying rather than trusting blindly.

Median imputation reappears here for the same statistical reason as Chapter 2: filling a skewed column's missing values with its mean would import the outlier-driven distortion directly into invented data points.

c. Python

`train_test_split` from scikit-learn is called twice in this project to carve three sets out of one DataFrame — once to separate out a test set, once again on the remainder to separate train from validation. `SimpleImputer(strategy='median')` is fit only on the training set and then applied to validation and test, which is what actually enforces the "no leakage" rule rather than just stating it. `StandardScaler` rescales numerical features to zero mean and unit variance — necessary for the logistic regression baseline, whose coefficients are sensitive to the scale of the underlying variable, but not needed for LightGBM in Chapter 6, since tree-based splits do not care about a feature's raw scale.

Try this yourself: If `train_test_split` were called without `stratify=y`, and validation AUC turned out slightly different across two different random seeds, what would that tell you about how much to trust a single validation split versus repeated cross-validation?

Chapter 5 — Baseline Model: Logistic Regression

a. Data Science Fundamentals

A baseline exists to answer one question honestly: is your more complex model actually earning its complexity? Without a baseline, a LightGBM AUC of 0.762 is just a number floating in space — you do not know if it is impressive or barely better than the simplest reasonable approach. Logistic regression is the standard baseline for binary classification specifically because its coefficients are directly readable: `logreg_coefficients.csv` shows AMT_GOODS_PRICE with a negative coefficient (about -0.44, the largest in magnitude) meaning higher goods price is associated with lower default risk in this linear model, while AMT_CREDIT has a positive coefficient (about +0.29). That direct readability is worth something real, even before you know whether the model performs well.

b. Statistics / Math

Logistic regression takes a weighted sum of input features (a linear combination, exactly like linear regression) and passes it through the logistic (sigmoid) function, which squashes any real number into a 0-1 range interpretable as a probability. The weighted sum itself is in “log-odds” units — this is why a coefficient’s sign tells you direction and its magnitude tells you strength, but not directly “how many percentage points,” which is a common misreading.

`class_weight='balanced'` changes how errors are penalized during training: instead of every misclassified row counting equally, misclassifying the minority class (defaulters, 8.07% of data) is weighted more heavily than misclassifying the majority class, roughly in inverse proportion to how rare

it is — which is why this project's baseline can achieve reasonable recall on defaults despite them being rare, rather than the model simply treating them as noise.

c. Python

`LogisticRegression(class_weight='balanced')` from scikit-learn implements everything above with one constructor argument. Once fit, `.coef_` exposes the learned weight for every feature in the order they were passed in, which is what produced logreg_coefficients.csv.

Try this yourself: EXT_SOURCE_MEAN has a coefficient of about -0.30 in this project's logistic regression. Given that EXT_SOURCE scores are external credit bureau scores where higher generally means lower risk, does that negative sign make intuitive sense? Why or why not?

Chapter 6 — Main Model: LightGBM

a. Data Science Fundamentals

Gradient boosting builds many small decision trees in sequence, where each new tree is trained specifically to correct the errors made by the trees before it, rather than each tree being built independently. This is why it typically outperforms logistic regression on tabular data like this project's: it captures nonlinear relationships and interactions between features automatically (income mattering differently depending on age, for instance) without you engineering every such interaction by hand the way DEBT_TO_INCOME was engineered in Chapter 3. The real trade-off, and the reason Chapter 8 (SHAP) exists in the project plan at all, is that this comes at the cost of the direct, per-coefficient interpretability logistic regression gave you for free.

b. Statistics / Math

scale_pos_weight is this project's LightGBM-specific way of achieving the same effect class_weight='balanced' achieved for logistic regression,

computed manually rather than through a named parameter: it is the ratio of negative-class count to positive-class count in the training data, which upweights the loss contribution from every defaulter roughly in proportion to how outnumbered they are.

Early stopping (stopping_rounds=50 in this project's code) addresses overfitting directly: LightGBM keeps adding trees only as long as validation performance keeps improving, and stops once 50 consecutive rounds pass without improvement — protecting against a model that keeps memorizing training-set noise after it has already learned everything generalizable.

c. Python

`lgb.train(params, train_data, num_boost_round=...)` is the core training call; `lgb.early_stopping(stopping_rounds=50)` is passed in as a callback that monitors validation performance during that training loop. The params dictionary (things like num_leaves, learning_rate) controls tree complexity and learning speed — worth noting honestly that this project used one fixed params dictionary rather than a tuned one, a real, identifiable gap rather than a hidden design choice.

Try this yourself: If this project's LightGBM had been trained with no early stopping at all — just a fixed, very high number of boosting rounds — what would you expect to happen to the gap between train_auc (0.826) and val_auc (0.762), and why?

Chapter 7 — Evaluation: AUC, Gini, AUC-PR, and KS

a. Data Science Fundamentals

Accuracy is close to meaningless here for the reason established back in Chapter 0: predicting "no default" unconditionally would already score 91.93% accuracy while catching zero actual defaulters. Every metric this project reports exists specifically to survive that trap. Looking at more than

one metric matters because each answers a subtly different question: AUC asks how well the model ranks risky applicants above safe ones across every possible cutoff, while AUC-PR asks the harsher question of, specifically among the applicants the model calls high-risk, how many actually were, and how many actual defaulters it caught — a distinction that matters more, not less, as the positive class gets rarer.

b. Statistics / Math

AUC-ROC is built from the ROC curve, which plots the true positive rate against the false positive rate as the decision threshold sweeps from 0 to 1 — the area under that curve is what gets reported, with 0.5 meaning no better than random and 1.0 meaning perfect separation. This project's validation AUC went from 0.745 (logistic regression) to 0.762 (LightGBM).

The Gini coefficient is a simple linear rescaling of AUC onto a -1 to 1 range (Gini = 2 x AUC - 1, so 0.745 becomes roughly 0.491 and 0.762 becomes roughly 0.523) — reported because it is the conventional metric in credit-risk industry practice, not because it says anything AUC does not already say.

AUC-PR (computed here via average_precision_score) summarizes the precision-recall trade-off across thresholds instead of the true/false-positive-rate trade-off, and is considered a more honest read than AUC-ROC specifically when positives are rare, because AUC-ROC can look deceptively strong even when precision on the rare class is poor.

The KS statistic — also present in this project's logged metrics (about 0.368 for the baseline, 0.388 for LightGBM) — measures the maximum gap between the cumulative score distributions of defaulters versus non-defaulters. A higher KS means the model separates the two groups more cleanly at whatever point that gap is largest, and it is another metric borrowed directly from credit-risk industry convention rather than general machine learning practice. Note: the metric values are confirmed real numbers from this project's own logged results — the exact line of code computing KS was not independently re-traced in this pass, so it is worth locating yourself as part of this chapter's exercise.

c. Python

`roc_auc_score(y_true, y_pred_proba)` computes AUC directly from true labels and predicted probabilities. `average_precision_score` computes AUC-PR the same way. `roc_curve` and `precision_recall_curve` return the raw points needed to actually plot either curve, rather than just the summary area underneath it — useful if you want to see where along the threshold range a model's advantage is concentrated.

Try this yourself: LightGBM's train AUC (0.826) is noticeably higher than its validation AUC (0.762), while logistic regression's train (0.748) and validation (0.745) AUC are nearly identical. What does that gap difference suggest about which model is more prone to overfitting on this data, and does that match Chapter 6's explanation of how each model works?

Chapter 8 — Interpretability: SHAP (genuinely not built yet)

a. Data Science Fundamentals

A model that performs well is not automatically a model you can deploy responsibly in credit scoring — a loan officer, a regulator, or a denied applicant may reasonably ask "why," and "the model said 0.83" is not an answer. There are two levels of explanation worth distinguishing: global explanation (which features matter most across all applicants, already partly available from feature_importance_final.csv) and individual explanation (why this specific applicant got this specific score) — SHAP is built specifically for the second, and it is the piece this project has scoped in the PRD but not yet built.

b. Statistics / Math

SHAP values come from Shapley values, a concept from cooperative game theory originally designed to fairly split a payoff among players who contributed unequally. Applied to machine learning, each feature is treated as a "player" contributing to the difference between an applicant's predicted score and the average predicted score, and the SHAP value

quantifies exactly how much that one feature pushed the prediction up or down for that one applicant.

Because LightGBM's raw output before the final transformation lives in log-odds space — the same units Chapter 5's logistic regression coefficients lived in — SHAP values are often reported in log-odds units too, and need converting back through the logistic function to be read as an actual probability shift. Worth knowing before a SHAP value looks confusingly unlike a plain percentage.

c. Python

`shap.TreeExplainer(model)` is built specifically for tree-based models like LightGBM, faster than the general-purpose explainer. Calling `.shap_values(X)` on it returns one contribution value per feature per row. From there, a waterfall plot breaks down one applicant's prediction feature by feature, while a summary plot shows global feature importance across many applicants at once — both draw on the same underlying calculation.

Try this yourself: Before writing any SHAP code, pick one real applicant row with a high EXT_SOURCE_MEAN and low DEBT_TO_INCOME. Predict, in your own words, roughly what you would expect their SHAP waterfall to show. Getting this prediction right — or seeing exactly where it is wrong — is the point of the exercise.

Chapter 9 — Dashboard: Streamlit (genuinely not built yet)

a. Data Science Fundamentals

The single most common way a real ML application breaks in production is not a bad model — it is train/serve skew: the preprocessing applied when the model was trained (imputation values, encoding mappings, engineered ratios) silently not matching what is applied when a new applicant is scored live. A dashboard that takes raw form inputs and shows a prediction has to

reproduce every transformation from Chapters 3 and 4 exactly, in the same order, using the same fitted objects (the same imputer, the same encoder mappings) — not reimplemented from scratch, which is a common and costly mistake.

b. Statistics / Math / Finance

A model outputs a probability, but a business needs a decision — approve, deny, or price the loan differently. That requires choosing a threshold (or several, for risk tiers), and that choice is a business and statistical trade-off, not a modeling one: a lower threshold catches more actual defaulters (higher recall) at the cost of denying more good applicants (lower precision), connecting directly back to the Type I/Type II error framing from Chapter 0's finance vocabulary and Chapter 7's metrics.

c. Python

Streamlit turns a Python script into an interactive web app with minimal boilerplate. Widgets like `st.number_input` or `st.selectbox` collect applicant details; `st.cache_data` and `st.cache_resource` avoid needlessly reloading the model or re-running expensive preprocessing on every interaction; and the app file's overall structure needs to mirror the notebook pipeline's order exactly — load the same fitted preprocessing objects, apply them in the same sequence, then call the model and, eventually, SHAP on the result.

Try this yourself: Sketch on paper, before writing any Streamlit code, the exact sequence of transformations a single raw applicant input would need to pass through, in order, to reach a final prediction. Compare that sequence against what Chapters 3 and 4 actually did.